

적절한 출처 표기가 이루어지는 경우, Google은 본 논문의 표와 그림을 언론 보도 또는 학술 연구 목적으로만 복제할 수 있는 권한을 부여합니다.

주의력만 있으면 충분하다

Ashish Vaswani* Google
Brain avaswani@google.com

노암 샤지르* 구글 브
레인 noam@google.com

니키 파르마* 구글 리서
치 nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

리온 존스* 구글 리서치
llion@google.com

에이든 N. 고메즈† 토론토 대
학교 aidan@cs.toronto.edu

루카시 카이저*
구글 브레인
lukaszkaizer@google.com

Illia Polosukhin*
illia.polosukhin@gmail.com

초록

현재 주류를 이루는 시퀀스 변환 모델들은 인코더와 디코더를 포함하는 복잡한 재귀 신경망이나 컨볼루션 신경망에 기반을 두고 있습니다. 성능이 가장 우수한 모델들은 또한 어텐션 메커니즘을 통해 인코더와 디코더를 연결합니다. 우리는 재귀 구조와 컨볼루션 연산을 완전히 배제하고 오로지 어텐션 메커니즘에만 기반을 둔 새로운 단순 네트워크 아키텍처인 ‘트랜스포머(Transformer)’를 제안합니다. 두 가지 기계 번역 과제에 대한 실험 결과, 본 모델은 품질 면에서 우수할 뿐만 아니라 병렬 처리가 용이하고 훈련 시간이 현저히 단축되는 것으로 나타났다. 본 모델은 WMT 2014 영어-독일어 번역 과제에서 28.4 BLEU 점수를 달성하여, 앙상블 모델을 포함한 기존 최고 기록을 2 BLEU 이상 상회했다. WMT 2014 영어-프랑스어 번역 과제에서, 본 모델은 8개의 GPU에서 3.5일간 훈련한 후 41.8 BLEU 점수를 기록하며 단일 모델 기준 새로운 최첨단 기록을 세웠으며, 이는 기존 문헌상의 최고 모델들에 비해 극히 적은 훈련 비용으로 달성된 결과이다. 또한, 대규모 및 제한된 훈련 데이터 모두를 사용하여 영어 구문 분석에 트랜스포머를 성공적으로 적용함으로써, 본 모델이 다른 과제에도 잘 일반화됨을 입증하였다.

* 동등한 기여. 나열 순서는 무작위입니다. 야콥(Jakob)은 RNN을 셀프 어텐션으로 대체할 것을 제안하고 이 아이디어를 평가하기 위한 작업을 시작했습니다. 아시시(Ashish)는 일리아(Illia)와 함께 최초의 트랜스포머 모델을 설계하고 구현했으며 이 작업의 모든 측면에 핵심적인 역할을 했습니다. 노암은 스케일링된 내적 어텐션(scaled dot-product attention), 멀티헤드 어텐션(multi-head attention), 그리고 매개변수 없는 위치 표현(parameter-free position representation)을 제안했으며, 거의 모든 세부 사항에 관여한 또 다른 인물이 되었습니다. 니키는 우리의 초기 코드베이스와 텐서2텐서(tensor2tensor)에서 수많은 모델 변형을 설계, 구현, 튜닝 및 평가했습니다. 리온 또한 새로운 모델 변형을 실험했으며, 초기 코드베이스와 효율적인 추론 및 시각화를 담당했습니다. 루카스와 에이단은 텐서2텐서의 다양한 부분을 설계하고 구현하는 데 수많은 긴 날을 보냈으며, 이를 통해 기존 코드베이스를 대체하고 결과를 크게 개선하며 연구 속도를 대폭 가속화했습니다.

† Google Brain 재직 당시 수행한 연구.

* Google Research 재직 당시 수행한 연구.

1 서론

재귀 신경망, 특히 장기 단기 기억(LSTM) [13] 및 게이트 재귀 신경망(GRN) [7]은 언어 모델링 및 기계 번역과 같은 시퀀스 모델링 및 변환 문제에서 최첨단 접근법으로 확고하게 자리 잡았습니다 [35, 2, 5]. 그 이후로 재귀 언어 모델과 인코더-디코더 아키텍처의 한계를 넓히기 위한 수많은 노력이 계속되어 왔다 [38, 24, 15].

재귀 모델은 일반적으로 입력 및 출력 시퀀스의 기호 위치에 따라 계산을 분할합니다. 위치들을 계산 시간의 단계와 일치시킴으로써, 이 모델들은 이전 숨겨진 상태 h_{t-1} 과 위치 t 에 대한 입력을 함수로 하는 숨겨진 상태 시퀀스 $h(t)$ 를 생성합니다. 이러한 본질적인 순차적 특성으로 인해 훈련 예제 내에서의 병렬 처리가 불가능하며, 메모리 제약으로 인해 예제 간 배치 처리가 제한되기 때문에 시퀀스 길이가 길어질수록 이는 심각한 문제가 됩니다. 최근 연구에서는 분해 기법[21]과 조건부 계산[32]을 통해 계산 효율성을 크게 향상시켰으며, 후자의 경우 모델 성능도 개선했다. 그러나 순차적 계산의 근본적인 제약은 여전히 남아 있다.

어텐션 메커니즘은 다양한 작업에서 강력한 시퀀스 모델링 및 변환 모델의 핵심 요소로 자리 잡았으며, 입력 또는 출력 시퀀스 내의 거리와 무관하게 의존성을 모델링할 수 있게 해줍니다 [2, 19]. 그러나 극히 일부 사례[27]를 제외하고는, 이러한 어텐션 메커니즘이 재귀 신경망과 함께 사용됩니다.

본 연구에서는 재귀 구조를 배제하고, 대신 어텐션 메커니즘에만 전적으로 의존하여 입력과 출력 간의 전역적 의존성을 도출하는 모델 아키텍처인 '트랜스포머(Transformer)'를 제안한다. 트랜스포머는 훨씬 더 높은 수준의 병렬 처리를 가능하게 하며, 8개의 P100 GPU에서 단 12시간만 훈련해도 번역 품질 측면에서 새로운 최첨단 수준에 도달할 수 있다.

2 배경

순차적 연산을 줄이려는 목표는 Extended Neural GPU [16], ByteNet [18], ConvS2S [9]의 기반이 되기도 하는데, 이들 모델은 모두 컨볼루션 신경망을 기본 구성 요소로 사용하여 모든 입력 및 출력 위치에 대해 숨겨진 표현을 병렬로 계산합니다. 이러한 모델에서, 두 임의의 입력 또는 출력 위치의 신호를 연관시키는 데 필요한 연산 횟수는 위치 간 거리에 따라 증가하는데, ConvS2S의 경우 선형적으로, ByteNet의 경우 대수적으로 증가합니다. 이로 인해 멀리 떨어진 위치 간의 의존성을 학습하기가 더 어려워집니다 [12]. Transformer에서는 이 연산 횟수가 일정한 수로 줄어들지만, 어텐션 가중치를 적용한 위치들을 평균화함으로써 유효 해상도가 저하되는 대가를 치르게 됩니다. 이러한 효과는 3.2절에서 설명한 바와 같이 Multi-Head Attention을 통해 상쇄합니다.

셀프 어텐션(self-attention)은 때때로 인트라 어텐션(intra-attention)이라고도 불리며, 단일 시퀀스의 표현을 계산하기 위해 시퀀스 내 서로 다른 위치들을 연관시키는 어텐션 메커니즘이다. 셀프 어텐션은 독해, 요약, 텍스트 합성, 과제 독립적 문장 표현 학습 등 다양한 과제에서 성공적으로 활용되어 왔다 [4, 27, 28, 22].

엔드투엔드 메모리 네트워크는 시퀀스 정렬 재귀 대신 재귀적 어텐션 메커니즘을 기반으로 하며, 단순 언어 질문 응답 및 언어 모델링 작업에서 우수한 성능을 보이는 것으로 입증되었다 [34].

그러나 우리가 아는 한, 트랜스포머(Transformer)는 시퀀스 정렬 RNN이나 컨볼루션(convolution)을 사용하지 않고 전적으로 셀프 어텐션을 통해 입력과 출력의 표현을 계산하는 최초의 전이(transduction) 모델이다. 다음 절에서는 트랜스포머를 설명하고, 셀프 어텐션의 필요성을 제시하며, [17, 18] 및 [9]와 같은 모델에 비해 갖는 장점을 논의할 것이다.

3 모델 아키텍처

가장 경쟁력 있는 신경망 시퀀스 변환 모델들은 대부분 인코더-디코더 구조를 갖는다 [5, 2, 35]. 여기서 인코더는 기호 표현의 입력 시퀀스 $(x_1, \dots, x_n)$ 를 연속적인 표현의 시퀀스 $\mathbf{z} = (z_1, \dots, z_n)$ 로 매핑한다. 주어진 $\mathbf{z}$ 를 바탕으로, 디코더는 기호로 구성된 출력 시퀀스 $(y_1, \dots, y_m)$ 를 한 요소씩 생성한다. 각 단계에서 모델은 자기회귀적(auto-regressive)이며 [10], 다음 기호를 생성할 때 이전에 생성된 기호들을 추가 입력으로 활용한다.

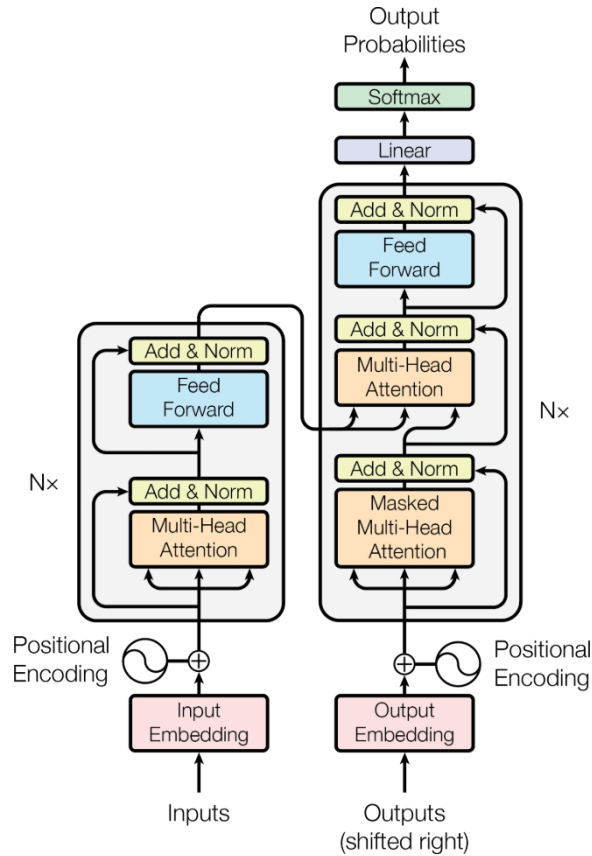


그림 1: 트랜스포머() - 모델 아키텍처.

트랜스포머는 인코더와 디코더 모두에 대해 스택형 자기 주의(self-attention)와 점별 완전 연결(point-wise, fully connected) 레이어를 사용하여 이러한 전반적인 아키텍처를 따르며, 이는 각각 그림 1의 왼쪽과 오른쪽 절반에 표시되어 있습니다.

3.1 인코더 및 디코더 스택

인코더: 인코더는 $N = 6$ 개의 동일한 레이어로 구성된 스택으로 이루어져 있습니다. 각 레이어는 두 개의 하위 레이어를 갖습니다. 첫 번째는 멀티헤드 자기 주의(self-attention) 메커니즘이고, 두 번째는 위치별 완전 연결 전방 전달(fully connected feed-forward) 네트워크입니다. 우리는 두 하위 레이어 각각에 잔차 연결[11]을 적용한 후, 레이어 정규화[1]를 수행합니다. 즉, 각 하위 레이어의 출력은 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 이며, 여기서 $\text{Sublayer}(x)$ 는 해당 하위 레이어 자체에서 구현된 함수입니다. 이러한 잔차 연결을 용이하게 하기 위해, 모델 내의 모든 하위 레이어와 임베딩 레이어는 차원 $d_{\text{model}} = 512$ 의 출력을 생성합니다.

디코더: 디코더 역시 $N = 6$ 개의 동일한 레이어로 구성된 스택으로 이루어져 있습니다. 각 인코더 레이어의 두 하위 레이어 외에도, 디코더는 인코더 스택의 출력에 대해 멀티헤드 어텐션을 수행하는 세 번째 하위 레이어를 삽입합니다. 인코더와 마찬가지로, 각 하위 레이어 주변에 잔차 연결을 적용한 후 레이어 정규화를 수행합니다. 또한 디코더 스택 내의 셀프 어텐션 서브레이어를 수정하여, 특정 위치가 그 이후의 위치에 주의를 기울이는 것을 방지합니다. 이러한 마스킹은 출력 임베딩이 한 위치만큼 오프셋된다는 사실과 결합되어, 위치 i 에 대한 예측이 위치 i 보다 작은 위치의 알려진 출력에만 의존하도록 보장합니다.

3.2 어텐션

어텐션 함수는 쿼리와 일련의 키-값 쌍을 출력으로 매핑하는 것으로 설명할 수 있으며, 여기서 쿼리, 키, 값 및 출력은 모두

벡터입니다. 출력은 가중치 합으로 계산됩니다

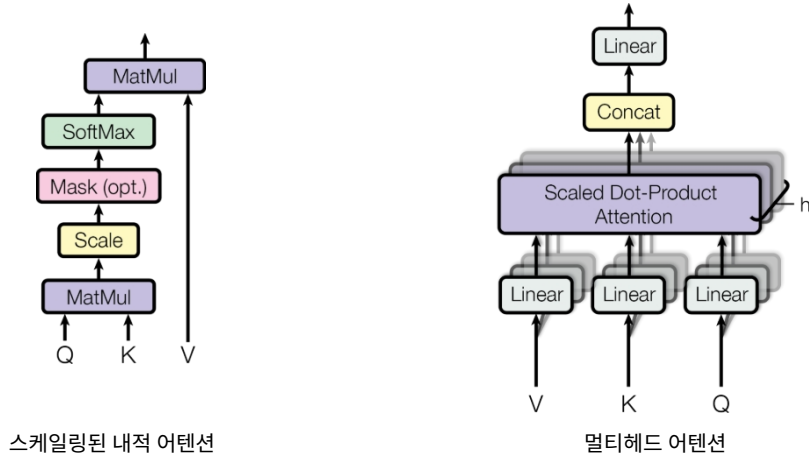


그림 2: (왼쪽) 스케일링된 내적 어텐션. (오른쪽) 멀티헤드 어텐션은 병렬로 실행되는 여러 어텐션 레이어로 구성됩니다.

각 값에 할당된 가중치는 쿼리와 해당 키 간의 호환성 함수를 통해 계산됩니다.

3.2.1 스케일링된 내적 어텐션

우리는 이 특정 어텐션을 "스케일링된 내적 어텐션"이라고 부릅니다(그림 2). 입력은 차원 d_k 의 쿼리와 키, 그리고 차원 d_v 의 값들로 구성됩니다. 우리는 모든 키를 포함한 쿼리의 내적을 계산하고, 각 값을 쿼리와 모든 키 간의 내적을 계산하고, 각각을 d_k 로 나눈 뒤, 소프트맥스 함수를 적용하여 값에 대한 가중치를 구합니다.

실제 구현에서는 쿼리 집합을 *행렬 Q*로 묶어 동시에 어텐션 함수를 계산합니다. 키와 값 또한 각각 행렬 *K*와 *V*로 묶습니다. 출력 행렬은 다음과 같이 계산합니다:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

가장 널리 사용되는 두 가지 어텐션 함수는 가산 어텐션[2]과 내적(곱셈) 어텐션입니다. 내적 어텐션은 $\sqrt{d_k}$ 의 스케일링 계수를 제외하면 본 알고리즘과 동일합니다. 가산 어텐션은

단일 숨겨진 층을 가진 피드포워드 네트워크를 사용하여 호환성 함수를 계산한다. 두 방식은 이론적 복잡도 면에서는 유사하지만, 내적 어텐션은 고도로 최적화된 행렬 곱셈 코드를 사용하여 구현할 수 있기 때문에 실제로는 훨씬 빠르고 공간 효율이 높다.

d_k 의 값이 작을 때는 두 메커니즘의 성능이 비슷하지만, d_k 의 값이 커질수록 스케일링을 적용하지 않은 내적 어텐션보다 가산 어텐션이 더 우수한 성능을 보입니다 [3]. 우리는 d_k 의 값이 클 경우 내적의 크기가 커지면서 소프트맥스 함수를 기울기가 극히 작은 영역으로 밀어넣는다고 추측한다⁴. 이러한 효과를 상쇄하기 위해, 우리는 내적에 $\sqrt{d_k}$ 를 곱하여 스케일링한다.

3.2.2 멀티헤드 어텐션

d_{model} 차원의 키, 값, 쿼리를 사용하여 단일 어텐션 함수를 수행하는 대신, 쿼리, 키, 값을 각각 서로 다른 학습된 선형 투영을 통해 d_k 차원, d_v 차원, d_q 차원으로 h 번 선형 투영하는 것이 효과적이라는 것을 발견했습니다. 그런 다음 이렇게 투영된 쿼리, 키, 값 각각에 대해 어텐션 함수를 병렬로 수행하여 d_v 차원의

⁴ 내적 값이 커지는 이유를 설명하기 위해, q 와 k 의 성분들이 서로 독립적인 확률 변수라고 가정하자. 평균 0, 분산 1인 변수들. 그러면 이들의 내적 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 는 평균 0, 분산 $d_{(k)}$ 가진다.

출력 값들. 이들은 연결된 후 다시 투영되어, 그림 2에 표시된 대로 최종 값을 산출한다.

다중 헤드 어텐션(Multi-head attention)을 사용하면 모델이 서로 다른 위치에 있는 서로 다른 표현 부분 공간의 정보에 동시에 주의를 기울일 수 있습니다. 단일 어텐션 헤드의 경우, 평균화 과정으로 인해 이러한 기능이 제한됩니다.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

여기서 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

투영 행렬이 매개변수 행렬 $W^Q \in \mathbb{R}^{d_{\text{모델}} \times d_k}$, $W^K \in \mathbb{R}^{d_{\text{모델}} \times d_k}$, $W^V \in \mathbb{R}^{d_{\text{모델}} \times d_v}$

그리고 $W^O \in \mathbb{R}^{h d_v \times d_{\text{모델}}}$.

본 연구에서는 $h = 8$ 개의 병렬 어텐션 레이어(헤드)를 사용합니다. 각 레이어에 대해 $d_k = d_v = d_{\text{모델}}/h = 64$ 를 적용합니다. 각 헤드의 차원이 축소됨에 따라, 전체 계산 비용은 전체 차원을 사용하는 단일 헤드 어텐션과 유사한 수준입니다.

3.2.3 본 모델에서의 어텐션 적용

Transformer는 다중 헤드 어텐션을 세 가지 방식으로 활용합니다:

- "인코더-디코더 어텐션" 레이어에서 쿼리는 이전 디코더 레이어에서 오며, 메모리 키와 값은 인코더의 출력에서 가져옵니다. 이를 통해 디코더의 모든 위치가 입력 시퀀스의 모든 위치에 주의를 기울일 수 있습니다. 이는 [38, 2, 9]와 같은 시퀀스-투-시퀀스 모델의 전형적인 인코더-디코더 어텐션 메커니즘을 모방합니다.
- 인코더에는 자기 주의(self-attention) 레이어가 포함되어 있습니다. 자기 주의 레이어에서는 모든 키, 값, 쿼리가 동일한 출처에서 비롯되는데, 이 경우 인코더의 이전 레이어 출력입니다. 인코더의 각 위치는 인코더의 이전 레이어에 있는 모든 위치에 주의를 기울일 수 있습니다.
- 마찬가지로, 디코더의 자기 주의(self-attention) 레이어는 디코더 내의 각 위치가 해당 위치까지(해당 위치 포함) 디코더 내의 모든 위치에 주의를 기울일 수 있도록 합니다. 자기 회귀적 특성을 유지하기 위해서는 디코더 내에서 정보가 왼쪽으로 흐르는 것을 방지해야 합니다. 우리는 스케일링된 내적 주의(scaled dot-product attention) 내부에서, 허용되지 않는 연결에 해당하는 소프트맥스(softmax) 입력의 모든 값을 마스킹(로 설정)함으로써 이를 구현합니다. 그림 2를 참조하십시오.

3.3 위치별 피드포워드 네트워크

어텐션 하위 레이어 외에도, 인코더와 디코더의 각 레이어에는 완전히 연결된 피드포워드 네트워크가 포함되어 있으며, 이는 각 위치에 대해 별도로 그리고 동일하게 적용됩니다. 이는 ReLU 활성화 함수를 중간에 둔 두 개의 선형 변환으로 구성됩니다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

선형 변환은 서로 다른 위치에서 동일하지만, 레이어마다 다른 매개변수를 사용합니다. 이를 다른 방식으로 설명하자면 커널 크기가 1인 두 개의 컨볼루션이라고 할 수 있습니다. 입력과 출력의 차원은 $d_{\text{모델}} = 512$ 이며, 내부 레이어의 차원은 $d_{\text{ff}} = 2048$ 입니다.

3.4 임베딩과 소프트맥스

다른 시퀀스 변환 모델과 마찬가지로, 우리는 학습된 임베딩을 사용하여 입력 토큰과 출력 토큰을 차원 $d_{\text{모델}}$ 의 벡터로 변환합니다. 또한 일반적인 학습된 선형 변환과 소프트맥스 함수를 사용하여 디코더의 출력을 예측된 다음 토큰 확률로 변환합니다.

본 모델에서는 두 임베딩 레이어와 사전소프트맥스
선형 변환 사이에 동일한 가중치 행렬을 공유하며, 이는 [30]과 유사합니다. 임베딩 레이어에서는 해당 가중치에 $d_{(\text{model})}$ 을 곱합
니다.

표 1: 다양한 레이어 유형에 따른 최대 경로 길이, 레이어별 복잡도 및 최소 연속 연산 횟수. 여기서 n 은 시퀀스 길이, d 는 표현 차원, k 는 컨볼루션의 커널 크기, r 은 제한적 자기 주의(restricted self-attention)에서 이웃 영역의 크기이다.

레이어 유형	레이어당 복잡도	순차적 연산	최대 경로 길이
자기 주의	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
재귀	$O(n \cdot d)$	$O(n)$	$O(n)$
컨볼루션	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
자기 주의(제한적)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 위치 인코딩

우리 모델에는 재귀 구조나 컨볼루션이 포함되어 있지 않으므로, 모델이 시퀀스의 순서를 활용할 수 있도록 하기 위해서는 시퀀스 내 토큰의 상대적 또는 절대적 위치에 대한 정보를 일부 주입해야 합니다. 이를 위해 인코더와 디코더 스택의 맨 아래에 있는 입력 임베딩에 “위치 인코딩”을 추가합니다. 위치 인코딩은 임베딩과 동일한 차원 d_{model} 을 가지므로, 두 요소를 합산할 수 있습니다. 학습된 방식과 고정된 방식 등 다양한 위치 인코딩 옵션이 존재합니다 [9].

이 연구에서는 서로 다른 주파수를 가진 사인 및 코사인 함수를 사용합니다:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{\text{model}}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{\text{model}}}) \end{aligned}$$

여기서 pos 는 위치이고 i 는 차원을 나타냅니다. 즉, 위치 인코딩의 각 차원은 하나의 사인파에 해당합니다. 파장은 2π 에서 $10000 \cdot 2\pi$ 까지 등차수열을 이룬다. 우리가 이 함수를 선택한 이유는, 고정된 오프셋 k 에 대해 $PE_{pos+k} \approx PE_{pos}$ 의 선형 함수로 표현될 수 있으므로, 모델이 상대적 위치를 통해 주의 집중을 쉽게 학습할 수 있을 것이라고 가정했기 때문이다.

또한 학습된 위치 임베딩[9]을 대신 사용하는 실험도 진행해 보았으며, 두 버전이 거의 동일한 결과를 산출한다는 사실을 확인했습니다(표 3의 (E) 행 참조). 우리는 사인파 버전을 선택했는데, 이는 모델이 훈련 과정에서 접했던 것보다 더 긴 시퀀스 길이로 결과를 외삽할 수 있게 해주기 때문입니다.

4 왜 셀프 어텐션인가

이 절에서는, 한 개의 가변 길이 기호 표현 시퀀스($x_1, \dots, x_n$)를 동일한 길이의 다른 시퀀스($z_1, \dots, z_n$)로 매핑하는 데 일반적으로 사용되는 재귀 및 컨볼루션 레이어와 비교하여, 셀프 어텐션 레이어의 다양한 측면을 살펴본다. 여기서 $x_i, z_i \in \mathbb{R}^d$ 이며, 이는 전형적인 시퀀스 변환 인코더나 디코더의 숨겨진 층과 유사합니다. 셀프 어텐션 사용을 정당화하기 위해 세 가지 요구 사항을 고려합니다.

하나는 레이어당 총 계산 복잡도입니다. 또 다른 하나는 필요한 순차 연산의 최소 개수로 측정되는, 병렬화할 수 있는 계산량입니다.

세 번째는 네트워크 내 장거리 의존성 간의 경로 길이입니다. 장거리 의존성을 학습하는 것은 많은 시퀀스 변환 작업에서 핵심적인 과제입니다. 이러한 의존성을 학습하는 능력에 영향을 미치는 핵심 요인 중 하나는 네트워크 내에서 신호가 전방 및 후방으로 이동해야 하는 경로의 길이입니다. 입력 및 출력 시퀀스의 임의의 위치 조합 간 경로가 짧을수록 장거리 의존성을 학습하기가 더 쉬워집니다[12]. 따라서 우리는 서로 다른 레이어 유형으로 구성된 네트워크에서 임의의 두 입력 및 출력 위치 간의 최대 경로 길이도 비교합니다.

표 1에서 언급된 바와 같이, 셀프 어텐션 레이어는 일정한 수의 순차적 연산을 통해 모든 위치를 연결하는 반면, 재귀 레이어는 $O(n)$ 의 순차적 연산을 필요로 합니다. 계산 복잡도 측면에서, 시퀀스

길이 n 의 표현 차원 d 보다 작을 때 **셀프 어텐션 레이어가 재귀 레이어보다** 빠르며, 이는 단어 단위(word-piece)와 같은 기계 번역 분야의 최첨단 모델에서 사용하는 문장 표현의 경우 대부분 해당되는 상황입니다.

[38] 및 바이트 페어 [31] 표현법. 매우 긴 시퀀스가 포함된 작업의 계산 성능을 향상시키기 위해, 셀프 어텐션을 해당 출력 위치를 중심으로 한 입력 시퀀스의 크기 r 의 근방 영역만 고려하도록 제한할 수 있다. 이렇게 하면 최대 경로 길이가 $O(n/r)$ 로 증가한다. 향후 연구에서 이 접근 방식을 더 깊이 탐구할 계획이다.

커널 너비 $k < n$ 의 단일 컨볼루션 레이어는 모든 입력-출력 위치 쌍을 연결하지 못합니다. 이를 위해서는 연속 커널의 경우 $O(n/k)$ 개의 컨볼루션 레이어를 쌓아야 하며, 확장 컨볼루션[18]의 경우 $O(\log_k(n))$ 개의 레이어를 쌓아야 하므로, 네트워크 내 임의의 두 위치 간 최장 경로의 길이가 증가하게 됩니다. 일반적으로 컨볼루션 레이어는 재귀 레이어보다 k 배 더 많은 연산 비용을 소모한다. 그러나 분리형 컨볼루션[6]은 복잡도를 $O(knd + nd^2)$ 로 상당히 낮춘다. 다만 $k = n$ 의 경우에도 분리형 컨볼루션의 복잡도는 자기 주의(self-attention) 레이어와 점별 피드포워드(point-wise feed-forward) 레이어를 결합한 것과 동일하며, 이는 본 모델에서 채택한 접근 방식이다.

부수적인 이점으로, 자기 주의(self-attention)는 더 해석 가능한 모델을 만들어낼 수 있습니다. 우리는 모델의 주의 분포를 조사하고, 부록에서 예시를 제시하며 논의합니다. 개별 주의 헤드(attention head)들이 서로 다른 작업을 수행하는 법을 명확히 학습할 뿐만 아니라, 많은 경우 문장의 구문적 및 의미적 구조와 관련된 행동을 보이는 것으로 보입니다.

5 훈련

이 섹션에서는 모델의 훈련 방식을 설명합니다.

5.1 훈련 데이터 및 배치

우리는 약 450만 개의 문장 쌍으로 구성된 표준 WMT 2014 영어-독일어 데이터셋을 사용하여 훈련을 수행했습니다. 문장은 약 37,000개의 토큰으로 구성된 원문-번역문 공통 어휘를 사용하는 바이트 페어 인코딩(byte-pair encoding) [3]을 통해 인코딩되었습니다. 영어-프랑스어의 경우, 3,600만 개의 문장으로 구성된 훨씬 더 방대한 WMT 2014 영어-프랑스어 데이터셋을 사용했으며, 토큰을 32,000개의 단어 단위 어휘로 분할했습니다 [38]. 문장 쌍은 대략적인 시퀀스 길이에 따라 묶여 배치되었습니다. 각 훈련 배치에는 약 25,000개의 원문 토큰과 25,000개의 대상 토큰을 포함하는 문장 쌍 세트가 포함되었습니다.

5.2 하드웨어 및 일정

우리는 NVIDIA P100 GPU 8개를 장착한 단일 시스템에서 모델을 훈련했습니다. 본 논문 전반에 걸쳐 설명된 하이퍼파라미터를 사용한 기본 모델의 경우, 각 훈련 단계당 소요 시간은 약 0.4초였습니다. 기본 모델은 총 100,000단계, 즉 12시간 동안 훈련되었습니다. 대형 모델(표 3의 맨 하단에 설명됨)의 경우 단계당 소요 시간은 1.0초였습니다. 대형 모델은 300,000단계(3.5일) 동안 훈련되었습니다.

5.3 최적화기

우리는 $\theta_1 = 0.9$, $\theta_2 = 0.98$, $\epsilon = 10^{-9}$ 로 설정된 Adam 최적화기[20]를 사용했다. 훈련 과정 중 학습률은 다음 공식에 따라 변화시켰다:

$$lrate = d_{\text{모델}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

이는 처음 $warmup_steps$ 훈련 단계 동안 학습률을 선형적으로 증가시키고, 그 이후에는 단계 번호의 역제곱근에 비례하여 감소시키는 것과 일치합니다. 우리는 $warmup_steps = 4000$ 을 사용했습니다.

5.4 정규화

훈련 중에 세 가지 유형의 정규화를 적용합니다:

표 2: 이 트랜스포머 모델은 영어-독일어 및 영어-프랑스어 newstest2014 테스트에서 기존 최첨단 모델들보다 더 높은

BLEU 점수를 기록했으며, 훈련 비용은 그 일부에 불과했습니다.

모델	BLEU		훈련 비용 (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{18}$
GNMT + RL [38]	24.6	39.92	2.3×10^{17}	1.4×10^{18}
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{17}$	$1.5 \cdot 10^{18}$
MoE [32]	26.03	40.56	2.0×10^{17}	1.2×10^{18}
Deep-Att + PosUnk 앙상블 [39]		40.4		$8.0 \cdot 10^{17}$
GNMT + RL 앙상블 [38]	26.30	41.16	1.8×10^{17}	1.1×10^{18}
ConvS2S 앙상블 [9]	26.36	41.29	7.7×10^{17}	1.2×10^{18}
트랜스포머 (기본 모델)	27.3	38.1	$3.3 \cdot 10^{18}$	
트랜스포머 (대형)	28.4	41.8	2.3×10^{17}	

잔차 드롭아웃: 각 하위 레이어의 출력이 하위 레이어 입력에 더해지고 정규화되기 전에 드롭아웃[33]을 적용합니다. 또한 인코더와 디코더 스택 모두에서 임베딩과 위치 인코딩의 합계에도 드롭아웃을 적용합니다. 기본 모델의 경우 $P_{drop} = 0.1$ 의 비율을 사용합니다.

라벨 스무딩 훈련 과정에서 $\epsilon_{ts} = 0.1$ [36] 값의 라벨 스무딩을 적용했습니다. 이는 모델이 불확실성을 더 많이 반영하도록 학습하게 하여 퍼플렉시티(perplexity)는 저하시키지만, 정확도와 BLEU 점수는 향상시킵니다.

6 결과

6.1 기계 번역

WMT 2014 영어-독일어 번역 과제에서, 대형 트랜스포머 모델(표 2의 Transformer (big))은 기존에 보고된 최고 성능 모델들(앙상블 포함)보다 2.0 BLEU 이상 높은 성적을 기록하며, 28.4라는 새로운 최첨단 BLEU 점수를 달성했습니다. 이 모델의 구성은 표 3의 맨 아래 행에 나와 있습니다. 8개의 P100 GPU를 사용하여 3.5일 동안 훈련을 수행했습니다. 심지어 우리의 기본 모델조차도 경쟁 모델들의 훈련 비용의 극히 일부만 소요되면서, 이전에 발표된 모든 모델과 앙상블을 능가합니다.

WMT 2014 영어-프랑스어 번역 과제에서, 우리의 대규모 모델은 BLEU 점수 41.0을 기록하며, 기존에 발표된 모든 단일 모델을 능가하는 성과를 보였으며, 이는 이전 최첨단 모델의 1/4 미만의 훈련 비용으로 달성된 결과입니다. 영어-프랑스어 번역을 위해 훈련된 Transformer(대형) 모델은 드롭아웃 비율 $P_{drop} = 0.3$ 대신 0.1을 사용했습니다.

기본 모델의 경우, 10분 간격으로 저장된 마지막 5개의 체크포인트를 평균하여 얻은 단일 모델을 사용했습니다. 대형 모델의 경우, 마지막 20개의 체크포인트를 평균했습니다. 빔 크기 4와 길이 페널티 $\alpha = 0.6$ 을 적용한 빔 검색(beam search)을 사용했습니다 [38]. 이러한 하이퍼파라미터는 개발 세트에 대한 실험을 거쳐 선정되었습니다. 추론 시 최대 출력 길이는 입력 길이 + 50으로 설정했으나, 가능한 경우 조기 종료하도록 했습니다 [38].

표 2는 본 연구의 결과를 요약하고, 본 모델의 번역 품질 및 훈련 비용을 문헌에 소개된 다른 모델 아키텍처와 비교한 내용을 담고 있다. 모델 훈련에 사용된 부동소수점 연산 횟수는 훈련 시간, 사용된 GPU 수, 그리고 각 GPU의 지속적 단정밀도 부동소수점 처리 능력 추정치^⑤를 곱하여 산출하였다.

6.2 모델 변형

트랜스포머의 다양한 구성 요소가 갖는 중요성을 평가하기 위해, 우리는 기본 모델을 여러 가지 방식으로 변형하고, 영어-독일어 번역 성능의 변화를 측정하였다.

⁵ K80, K40, M40, P100에 대해 각각 2.8, 3.7, 6.0, 9.5 TFLOPS의 값을 사용했습니다.

표 3: 트랜스포머 아키텍처의 변형. 나열되지 않은 값은 기본 모델의 값과 동일합니다. 모든 지표는 영어-독일어 번역 개발 데이터셋인 newstest2013을 기준으로 합니다. 나열된 퍼플렉시티는 당사의 바이트 쌍 인코딩에 따라 단어 조각(wordpiece) 당 값이며, 단어당 퍼플렉시티와 비교해서는 안 됩니다.

	N	d_{model}	$diff$	h	d_k	d_v	P_{drop}	ϵ_{ls}	훈련 단계	PPL (dev)	BLEU (개발)	매개변수 $\times 10^6$		
기본값	6	512	2048	8	64	64	0.1	0.1	10만	4.92	25.8	65		
(A)					1	512	512				5.29	24.9		
					4	128	128				5.00	25.5		
					16	32	32				4.91	25.8		
					32	16	16				5.01	25.4		
(B)					16					5.16	25.1	58		
					32					5.01	25.4	60		
(C)	2									6.11	23.7	36		
	4									5.19	25.3	50		
	8									4.88	25.5	80		
		256			32	32					5.75	24.5	28	
		1024			128	128					4.66	26.0	168	
			1024									5.12	25.4	53
			4096									4.75	26.2	90
(D)							0.0				5.77	24.6		
							0.2				4.95	25.5		
								0.0		4.67	25.3			
								0.2		5.47	25.7			
(E)	사인파 대신 위치 임베딩									4.92	25.7			
big	6	1024	4096	16				0.3	300K	4.33	26.4	213		

개발 세트, newstest2013. 이전 섹션에서 설명한 대로 빔 검색을 사용했으나, 체크포인트 평균화는 적용하지 않았습니다. 이 결과들은 표 3에 제시합니다.

표 3의 (A) 행에서는 3.2.2절에서 설명한 바와 같이, 계산량을 일정하게 유지한 상태에서 어텐션 헤드 수와 어텐션 키 및 값의 차원을 변화시켜 보았다. 단일 헤드 어텐션은 최상의 설정보다 BLEU 점수가 0.9점 낮지만, 헤드 수가 너무 많을 경우에도 품질이 떨어지는 것으로 나타났다.

표 3의 (B)행에서는 어텐션 키 크기 d_k 를 줄이면 모델 품질이 저하되는 것을 관찰할 수 있습니다. 이는 호환성을 판단하는 것이 쉽지 않으며, 내적(dot product)보다 더 정교한 호환성 함수를 사용하는 것이 유리할 수 있음을 시사합니다. 또한 (C) 및 (D) 행에서 예상대로 모델 규모가 클수록 성능이 우수하며, 드롭아웃(dropout)이 과적합을 방지하는 데 매우 효과적임을 확인할 수 있다. (E) 행에서는 사인파형 위치 인코딩을 학습된 위치 임베딩[9]으로 대체하였으며, 기본 모델과 거의 동일한 결과를 얻었다.

6.3 영어 구성 요소 구문 분석

트랜스포머가 다른 작업으로 일반화될 수 있는지 평가하기 위해, 우리는 영어 구성 요소 구문 분석에 대한 실험을 수행했다. 이 작업은 몇 가지 특정한 난제를 안고 있다. 즉, 출력은 강력한 구조적 제약을 받으며 입력보다 훨씬 길다. 게다가 RNN 시퀀스-투-시퀀스 모델은 소규모 데이터 환경에서는 최첨단 성능을 달성하지 못했다 [37].

우리는 Penn Treebank [25]의 월스트리트 저널(WSJ) 부분(약 4만 개의 훈련 문장)을 사용하여 $d_{model}=1024$ 인 4층 트랜스포머를 훈련했다. 또한 약 1,700만 개의 문장으로 구성된 더 큰 규모의 high-confidence 및 BerkleyParser 코퍼스 [37]를 사용하여 반감독 학습 환경에서도 모델을 훈련했다. WSJ 전용 설정에는 16,000개의 토큰으로 구성된 어휘를, 반감독 학습 설정에는 32,000개의 토큰으로 구성된 어휘를 사용했다.

우리는 드롭아웃(attention 및 residual 모두, 5.4절), 학습률, 빔 크기를 선정하기 위해 22절의 개발 데이터셋에서 소수의 실

험만을 수행했으며, 그 외 모든 파라미터는 영어-독일어 기본 번역 모델과 동일하게 유지했습니다. 추론 단계에서는, 우리는

표 4: 트랜스포머는 영어 구문 분석에 잘 일반화된다(결과는 WSJ의 23절에 있음)

구문 분석기	훈련	WSJ 23 F1
Vinyals & Kaiser 외 (2014) [37]	WSJ 전용, 판별적	88.3
Petrov 외 (2006) [29]	WSJ 전용, 판별적	90.4
Zhu 외 (2013) [40]	WSJ 전용, 판별적	90.4
Dyer 외 (2016) [8]	WSJ 전용, 판별적	91.7
트랜스포머 (4층)	WSJ 전용, 판별 학습	91.3
Zhu 외 (2013) [40]	반감독 학습	91.3
Huang & Harper (2009) [14]	반감독 학습	91.3
McClosky 외 (2006) [26]	반지도 학습	92.1
Vinyals & Kaiser 외 (2014) [37]	반감독 학습	92.1
트랜스포머 (4층)	반감독 학습	92.7
Luong 외 (2015) [23]	다중 작업	93.0
Dyer 외 (2016) [8]	생성적	93.3

최대 출력 길이를 입력 길이 + 300으로 늘렸다. WSJ 전용 및 반감독 학습 설정 모두에서 빔 크기 21과 $\alpha = 0.3$ 을 사용했다.

표 4의 결과에서 볼 수 있듯이, 특정 작업에 대한 튜닝이 이루어지지 않았음에도 불구하고 우리 모델은 놀라울 정도로 우수한 성능을 보였으며, 재귀 신경망 문법(Recurrent Neural Network Grammar) [8]을 제외한 이전에 보고된 모든 모델보다 더 나은 결과를 산출했습니다.

RNN 시퀀스-투-시퀀스 모델[37]과 달리, Transformer는 4만 개의 문장으로 구성된 WSJ 훈련 세트만으로 훈련한 경우에도 Berkeley-Parser[29]보다 우수한 성능을 보입니다.

7 결론

이 연구에서 우리는 엔코더-디코더 아키텍처에서 가장 일반적으로 사용되는 재발 레이어를 다중 헤드 자기 주의(multi-headed self-attention)로 대체한, 전적으로 주의(attention)에 기반한 최초의 시퀀스 변환 모델인 Transformer를 제시했다.

번역 작업의 경우, Transformer는 재귀 또는 컨볼루션 레이어를 기반으로 하는 아키텍처보다 훨씬 빠르게 훈련될 수 있습니다. WMT 2014 영어-독일어 및 WMT 2014 영어-프랑스어 번역 작업 모두에서 우리는 새로운 최첨단 성과를 달성했습니다. 전자의 작업에서 우리의 최우수 모델은 이전에 보고된 모든 양상을 모델들보다 뛰어난 성능을 보였습니다.

우리는 어텐션 기반 모델의 미래에 큰 기대를 걸고 있으며, 이를 다른 과제에도 적용할 계획입니다. 텍스트 이외의 입력 및 출력 모달리티를 다루는 문제로 트랜스포머(Transformer)를 확장하고, 이미지, 오디오, 비디오와 같은 대규모 입력 및 출력을 효율적으로 처리하기 위한 국소적이고 제한적인 어텐션 메커니즘을 연구할 예정입니다. 또한, 생성 과정을 덜 순차적으로 만드는 것 역시 우리의 또 다른 연구 목표입니다.

모델 훈련 및 평가에 사용한 코드는 <https://github.com/tensorflow/tensor2tensor>에서 확인할 수 있습니다.

감사의 말: 유익한 의견, 수정 사항 및 영감을 주신 Nal Kalchbrenner와 Stephan Gouws에게 깊은 감사를 드립니다.

참고문헌

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E Hinton. Layer normalization. *arXiv 사전 인쇄본 arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio. 정렬과 번역을 동시에 학습하는 신경망 기계 번역. *CoRR*, abs/1409.0473, 2014.
- [3] 데니 브리츠, 안나 골디, 민-탕 루옹, 환 V. 레. 신경망 기계 번역 아키텍처에 대한 대규모 탐구. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, 및 Mirella Lapata. 기계 독해를 위한 장기 단기 기억(LSTM) 네트워크. *arXiv 사전 인쇄본 arXiv:1601.06733*, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, 및 Yoshua Bengio. 통계적 기계 번역을 위한 RNN 인코더-디코더를 이용한 구문 표현 학습. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: 깊이별 분리 가능한 컨볼루션(depthwise separable convolutions)을 활용한 딥 러닝. *arXiv 프리프린트 arXiv:1610.02357*, 2016.
- [7] 정준영, 차글라르 굴체레, 조경현, 요슈아 벙지오. 시퀀스 모델링에 대한 게이트 재귀 신경망의 경험적 평가. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, 및 Noah A. Smith. 재귀 신경망 문법. *NAACL 회의록*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, 및 Yann N. Dauphin. 컨볼루션 시퀀스-투-시퀀스 학습. *arXiv 사전 인쇄본 arXiv:1705.03122v2*, 2017.
- [10] 알렉스 그레이브스. 재귀 신경망을 이용한 시퀀스 생성. *arXiv 사전 인쇄본 arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, 및 Jian Sun. 이미지 인식을 위한 딥 잔차 학습. *IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스 논문집*, 770–778쪽, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, 및 Jürgen Schmidhuber. 재귀 신경망에서의 기울기 흐름: 장기적 의존성 학습의 어려움, 2001.
- [13] Sepp Hochreiter 및 Jürgen Schmidhuber. 장기 단기 기억(Long Short-Term Memory). *Neural Computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang 및 Mary Harper. 다국어 간 잠재적 주석을 활용한 PCFG 문법의 자가 학습. *2009년 자연어 처리 경험적 방법 컨퍼런스(Conference on Empirical Methods in Natural Language Processing) 논문집*, 832–841쪽. ACL, 2009년 8월.
- [15] 라팔 요제포비치, 오리올 비난스, 마이크 슈스터, 노암 샤지르, 옹후이 우. 언어 모델링의 한계 탐구. *arXiv 사전 인쇄본 arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser 및 Samy Bengio. 능동적 기억이 어텐션을 대체할 수 있을까? *신경 정보 처리 시스템의 발전(NIPS)*, 2016.
- [17] Łukasz Kaiser 및 Ilya Sutskever. 신경망 GPU는 알고리즘을 학습한다. *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, Ko-ray Kavukcuoglu. 선형 시간 내 신경망 기계 번역. *arXiv 사전 인쇄본 arXiv:1610.10099v2*, 2017.
- [19] 윤 김, 칼 덴튼, 루옹 호앙, 알렉산더 M. 러시. 구조화된 어텐션 네트워크. *국제 학습 표현 컨퍼런스(ICLR)*, 2017.
- [20] 디데릭 킹마, 지미 바. Adam: 확률적 최적화를 위한 방법. *ICLR*, 2015.
- [21] 올렉시 쿠차예프 및 보리스 긴스버그. LSTM 네트워크를 위한 분해 기법. *arXiv 사전 인쇄본 arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, Yoshua Bengio. 구조화된 자기 주의형 문장 임베딩. *arXiv 사전 인쇄본 arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, 및 Łukasz Kaiser. 다중 작업 시퀀스-투-시퀀스 학습. *arXiv 사전 인쇄본 arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, 및 Christopher D Manning. 어텐션 기반 신경망 기계 번역을 위한 효과적인 접근법.

arXiv 사전 인쇄본 *arXiv:1508.04025*, 2015.

- [25] 미첼 P. 마커스, 메리 앤 마르친키에비치, 베아트리체 산토리니. 대규모 영어 주석 코퍼스 구축: 펜 트리뱅크. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, Mark Johnson. 구문 분석을 위한 효과적인 자가 학습. *NAACL 인간 언어 기술 컨퍼런스(Human Language Technology Conference) 논문집, 본 회의*, 152–159쪽. ACL, 2006년 6월.
- [27] 안쿠르 파리크, 오스카르 텍스트룸, 디판잔 다스, 야콥 우스코레이트. 분해 가능한 어텐션 모델. *《자연어 처리의 경험적 방법》*, 2016.
- [28] 로맹 파울루스, 카이밍 송, 리처드 소처. 요약 생성을 위한 딥 강화 학습 모델. *arXiv 사전 인쇄본 arXiv:1705.04304*, 2017.
- [29] 슬라브 페트로프, 레온 배럿, 로맹 티보, 댄 클라인. 정확하고 간결하며 해석 가능한 트리 어노테이션 학습. *제21회 국제 컴퓨터 언어학 컨퍼런스 및 제44회 ACL 연례 회의 논문집*, 433–440쪽. ACL, 2006년 7월.
- [30] 오피르 프레스(Ofir Press) 및 리오르 울프(Lior Wolf). 출력 임베딩을 활용한 언어 모델 개선. *arXiv 사전 인쇄본 arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, Alexandra Birch. 서브워드 단위를 활용한 희귀 단어의 신경망 기계 번역. *arXiv 사전 인쇄본 arXiv:1508.07909*, 2015.
- [32] 노암 샤지르, 아잘리아 미르호세이니, 크시슈토프 마지아르즈, 앤디 데이비스, 환 레, 제프리 힌턴, 제프 딘. 엄청난게 큰 신경망: 스파스 게이트 혼합 전문가 층. *arXiv 사전 인쇄본 arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, 및 Ruslan Salakhutdinov. Dropout: 신경망의 과적합을 방지하는 간단한 방법. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, 및 Rob Fergus. 종단 간 메모리 네트워크. C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, 및 R. Garnett 편집, *Advances in Neural Information Processing Systems* 28, 2440–2448쪽. Curran Associates, Inc., 2015.
- [35] 일리아 수츠케버, 오리올 비난스, 환 V. V. 레. 신경망을 이용한 시퀀스-투-시퀀스 학습. 『*신경 정보 처리 시스템의 발전(Advances in Neural Information Processing Systems)*』, 3104–3112쪽, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, 및 Zbigniew Wojna. 컴퓨터 비전을 위한 인셉션 아키텍처 재고. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, Hinton. 외국어로서의 문법. 『*신경 정보 처리 시스템의 발전(Advances in Neural Information Processing Systems)*』, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey 외. Google의 신경망 기계 번역 시스템: 인간 번역과 기계 번역의 격차 해소. *arXiv 사전 인쇄본 arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, Wei Xu. 신경망 기계 번역을 위한 패스트포워드 연결을 갖춘 심층 재귀 모델. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, 및 Jingbo Zhu. 빠르고 정확한 시프트-리듀스 구성 요소 구문 분석. *제51회 ACL 연례 회의 논문집(제1권: 장문 논문)*, 434–443쪽. ACL, 2013년 8월.

어텐션 시각화

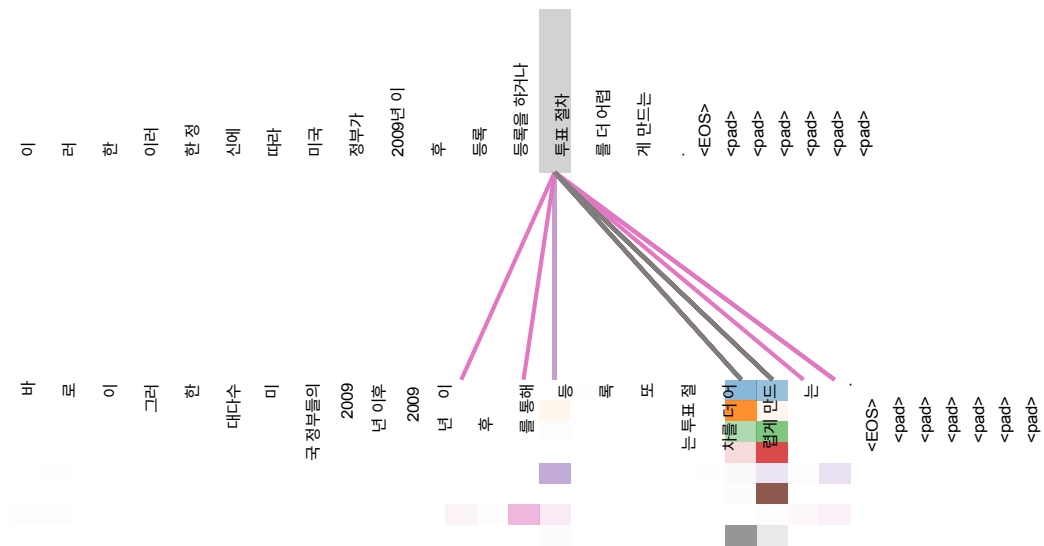


그림 3: 6층 중 5층의 인코더 자기 주의(self-attention)에서 장거리 의존성을 따르는 주의 메커니즘의 예시. 많은 주의 헤드(attention head)가 동사 ‘making’의 먼 의존성에 주의를 기울여 ‘making...more difficult’라는 구문을 완성한다. 여기서는 ‘making’이라는 단어에 대한 주의만 표시되어 있다. 서로 다른 색상은 서로 다른 헤드를 나타낸다. 컬러로 볼 때 가장 잘 보인다.

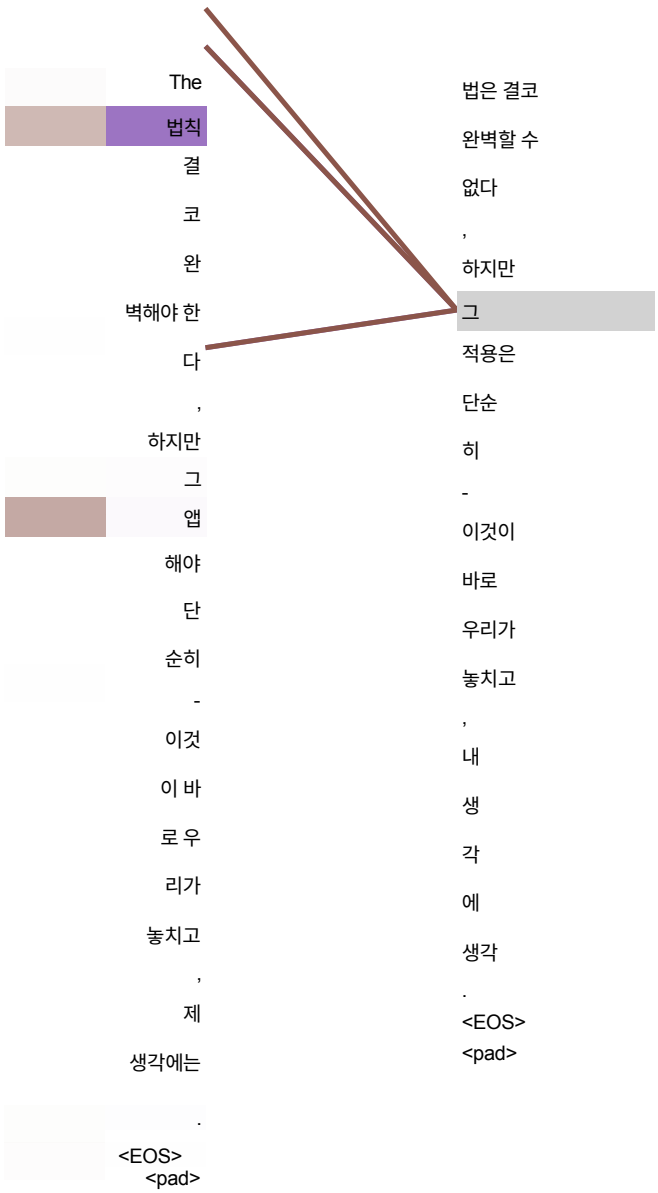
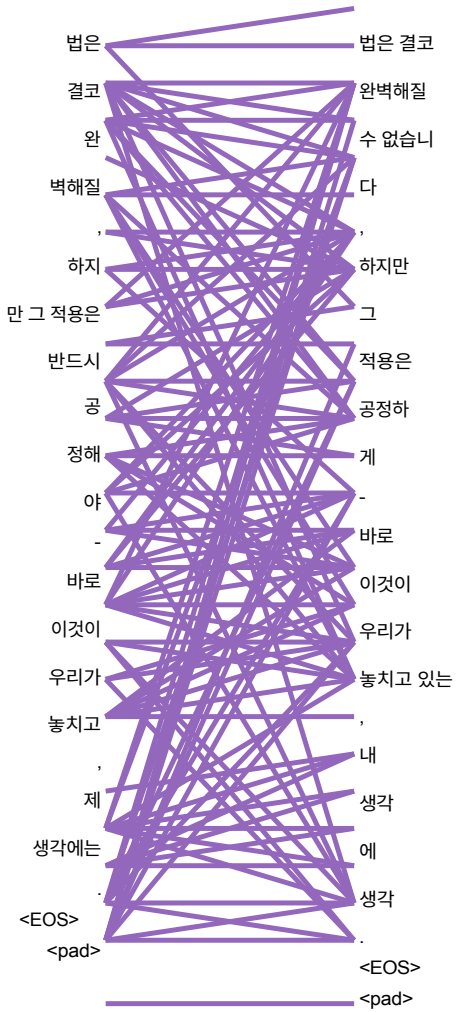


그림 4: 6개 레이어 중 5번째 레이어에 위치한 두 개의 어텐션 헤드. 이 헤드들은 지시어 해결에 관여하는 것으로 보인다. 위: 헤드 5에 대한 전체 어텐션. 아래: 어텐션 헤드 5와 6에 대해 'is'라는 단어에서만 추출한 어텐션. 이 단어에 대한 어텐션이 매우 선명하다는 점에 유의하라.



그림 5: 많은 어텐션 헤드가 문장의 구조와 관련된 것으로 보이는 행동을 보인다. 위에는 6층 중 5층에 위치한 인코더 자기 어텐션의 서로 다른 두 헤드에서 가져온 두 가지 예시를 제시한다. 이 헤드들은 서로 다른 작업을 수행하도록 명확히 학습되었다.